

WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Aiden Ward

Email: ward3aj@mail.uc.edu

Short-bio: Aiden Ward has a lot of interest in computer science, specifically data science. I also love volleyball and cars.



Figure 1: Aiden's headshot

Repository Information

Repository's URL: <https://github.com/aidenward6/waph-ward3aj/tree/main/waph-ward3aj/labs/project2>

Overview

For this project I extended my login system from Lab 3 and 4 into a full secure web application. The goal was to build out registration, profile view/edit, and password management on top of the session handling and prepared statements I already had working. This gave me a chance to bring together everything I learned this semester, from input validation to session security to CSRF protection, into one full application instead of separate lab exercises.

Requirement 1: User Registration

Registration is split across two pages. `registrationform.php` is the front-end form with username, name, email, password, and confirm password fields, and it

uses HTML5 `required/pattern/title` attributes so the browser catches bad input before it's ever sent, like a weak password or a username with spaces in it. There's also a small JS function, `checkPasswordsMatch()`, that blocks the submit if the two password fields don't match.

`addnewuser.php` is the back-end that actually handles it. It re-validates everything server-side with `isValidUsername()`, `isValidPassword()`, and `isValidEmail()` since you should never trust the client, sanitizes the inputs with `trim stripslashes/htmlspecialchars`, checks for a duplicate username with a prepared `SELECT`, and then hashes the password with `password_hash()` before inserting the new row with a prepared `INSERT`.

Requirement 2: Login

Login lives in `index.php`. It sets secure session cookie params (`secure`, `httponly`, `samesite=Strict`) before `session_start()` even runs, then on POST it looks up the user with a prepared `SELECT`, and checks the submitted password against the stored hash with `password_verify()`. If it matches, I call `session_regenerate_id(true)` to get a fresh session ID, mark the session as authenticated, store the username and the browser's User-Agent, and redirect to the profile page.

Requirement 3: Profile Management

`profile.php` requires `session_auth.php` at the top, so only logged in users can even reach it. It pulls the user's name and email out of the database with a prepared `SELECT` and pre-fills the edit form with them. When the form is submitted, it checks the CSRF token first, then validates and sanitizes the new name/email, then runs a prepared `UPDATE` and shows a success message.

Requirement 4: Password Update

`changepasswordform.php` generates a CSRF token and shows a form for the new password plus a confirm field, with the same strength requirements as registration, again backed up by a JS match check. `changepassword.php` verifies the CSRF token, re-validates the new password server-side, hashes it with `password_hash()`, and updates it with a prepared `UPDATE`. It's important that this uses the username stored in the session, not anything from user input, so a logged-in user can only ever change their own password.

Security: HTTPS, Hashed Passwords, No Root Account, Prepared Statements

The whole app is served over HTTPS with a self-signed cert, same setup as Lab 4, and every session cookie is marked `secure` so it will never get sent in plaintext. Passwords are never stored in plaintext, I use `password_hash()`

(bcrypt) on registration and password change, and `password_verify()` to check them on login. The database connection in `db_connect.php` uses the non-root `ward3aj` MySQL account from Lab 3, which only has access to the `waph` database and nothin else. And every single SQL query in the project, from login to registration to the profile/password updates, goes through a prepared statement with `bind_param()`, there's no raw string concatenation into SQL anywhere in the codebase.

Input Validation

Every form field has client-side validation through HTML5 `required` and `pattern` attributes, plus a `title` so the browser shows a helpful message. On the server side, `addnewuser.php` has dedicated `isValidUsername()`, `isValidPassword()`, and `isValidEmail()` functions that mirror the same rules, plus `profile.php` and `changepassword.php` re-check name/email/password server-side too. All inputs get sanitized with `trim`, `stripslashes`, and `htmlspecialchars` before they touch the database or get echoed back to the page, since client-side checks can always be bypassed by an attacker going straight to the PHP endpoint.

Database Design

The `users` table has `username` (varchar 50, primary key), `password` (varchar 255), `name` (varchar 100), and `email` (varchar 100, unique). I had to drop the old Lab 3 table and make a separate one with a wider password column, since bcrypt hashes from `password_hash()` are way longer than the old MD5 hashes and won't fit in the smaller column, plus MD5 hashes can't be checked with `password_verify()` anyway. All of this is set up in `database-data.sql`, which also seeds two test accounts.

Front-end Development

I reused and extended the CSS from Individual Project 1 (`style.css`), so all the forms (login, register, profile, change password) share the same look, including dark mode support. All the forms use HTML5 client-side validation with `required/pattern/title`, and I added small vanilla JS for the password-confirmation checks. Didn't think this project needed a full front-end framework given the scope.

Session Management

`session_auth.php` is a shared auth guard that gets `require'd` at the top of every protected page (`profile.php`, `changepasswordform.php`, `changepassword.php`), so I only have to write the login check once instead of copy-pasting it everywhere. It sets the same secure cookie params as `index.php`, and on login I call `session_regenerate_id(true)` to prevent session fixation.

To help protect against session hijacking, the User-Agent is stored in the session at login and compared against the current request on every protected page. If they don't match, the session is destroyed and the user gets bounced back to login.

CSRF Protection

Both `profile.php` and `changepasswordform.php` generate a random token with `openssl_random_pseudo_bytes(16)`, store it in `$_SESSION['csrf_token']`, and embed it in the form as a hidden input. When the form is submitted, the token from the POST data is compared to the one in the session using `hash_equals()`, a timing-safe comparison, and if they don't match the request is rejected before any database update happens. This stops an attacker from tricking a logged-in user into submitting a malicious form from another site.

Demonstration Video

Demo Video

Appendix: Source Code

index.php

```
<?php
// cookie setup
session_set_cookie_params([
    'lifetime' => 1800,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

session_start();

if (isset($_SESSION['authenticated']) && $_SESSION['authenticated'] === true)
{
    header("Location: profile.php");
    exit();
}

$error = "";

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    require 'db_connect.php';
```

```

$username = trim(stripslashes($_POST['username'] ?? ''));
$password = $_POST['password'] ?? '';

// double check, though html already checks it
if (!preg_match('/^[a-zA-Z0-9_]{3,50}$/', $username) || $password === '') {
    $error = 'Invalid username or password.';
} else {
    $stmt = $mysqli->prepare("SELECT password FROM users WHERE username = ?");
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $stmt->bind_result($hashed_password);

    if ($stmt->fetch() && password_verify($password, $hashed_password)) {
        $stmt->close();
        // new session id every login, learned this from lab4 hijac
        session_regenerate_id(true);
        $_SESSION['authenticated'] = true;
        $_SESSION['username'] = $username;
        $_SESSION['user_agent'] = $_SERVER['HTTP_USER_AGENT'];
        header("Location: profile.php");
        exit();
    } else {
        $stmt->close();
        $error = 'Invalid username or password.';
    }
}
$mysqli->close();
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WAPH - Login</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>WAPH Project 2 - Login</h1>

        <?php if ($error !== ''): ?>
            <p class="error"><?php echo htmlspecialchars($error, ENT_QUOTES, 'UTF-8'); ?></p>
        <?php endif; ?>
        <?php if (isset($_GET['registered'])): ?>
            <p class="success">Registration successful. Please log in.</p>

```

```

<?php endif; ?>

<form method="POST" action="index.php" class="form login">
    <label>Username:
        <input type="text" class="text_field" name="username"
            required maxlength="50" pattern="[a-zA-Z0-9_]{3,50}"
            title="3-50 characters: letters, numbers, underscore only">
    </label>
    <label>Password:
        <input type="password" class="text_field" name="password" required>
    </label>
    <button class="button" type="submit">Login</button>
</form>

<p>Don't have an account? <a href="registrationform.php">Register here</a>.</p>
</div>
</body>
</html>

```

registrationform.php

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WAPH - Register</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>WAPH Project 2 - Register</h1>

        <!-- reused the css from project 1, form checks passwords match before sending -->
        <form method="POST" action="addnewuser.php" class="form registration" onsubmit="return validateForm()">
            <label>Username:
                <input type="text" class="text_field" name="username" id="username"
                    required maxlength="50" pattern="[a-zA-Z0-9_]{3,50}"
                    title="3-50 characters: letters, numbers, underscore only">
            </label>
            <label>Name:
                <input type="text" class="text_field" name="name" id="name"
                    required maxlength="100">
            </label>
            <label>Email:
                <input type="email" class="text_field" name="email" id="email"
                    required maxlength="100">
            </label>
        </form>
    </div>

```

```

</label>
<!-- gotta have upper lower number and a symbol or it wont let u submit -->
<label>Password:
    <input type="password" class="text_field" name="password" id="password"
        required maxlength="50"
        pattern="(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[a-zA-Z0-9]).{8,50}"
        title="At least 8 characters, with 1 uppercase, 1 lowercase, 1 number"
    </input>
</label>
<label>Confirm Password:
    <input type="password" class="text_field" name="confirm_password" id="confirm_password"
        required maxlength="50"
    </input>
</label>
<p class="error" id="match-error" style="display:none;">Passwords do not match.</p>
<button class="button" type="submit">Register</button>
</form>

<p>Already have an account? <a href="index.php">Login here</a>.</p>
</div>

<script>

    function checkPasswordsMatch() {
        const password = document.getElementById('password').value;
        const confirm = document.getElementById('confirm_password').value;
        const errorEl = document.getElementById("match-error");
        if (password !== confirm) {
            errorEl.style.display = 'block';
            return false;
        }
        errorEl.style.display = 'none';
        return true;
    }
</script>
</body>
</html>

```

addnewuser.php

```

<?php
require 'db_connect.php';

// just some regex checks
function isValidUsername($username) {
    return preg_match('/^[a-zA-Z0-9_]{3,50}$/', $username) === 1;
}

```

```

function isValidPassword($password) {
    return preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[^\a-zA-Z0-9]).{8,50}$/', $password);
}

function isValidEmail($email)
{
    return filter_var($email, FILTER_VALIDATE_EMAIL) !== false && strlen($email) <= 100;
}

$errors = array();

// cleaning up everything before it touches the page or the db
$username = htmlspecialchars(trim(stripslashes($_POST['username']) ?? ''), ENT_QUOTES, 'UTF-8');
$name = htmlspecialchars(trim(stripslashes($_POST['name']) ?? ''), ENT_QUOTES, 'UTF-8');
$email = htmlspecialchars(trim(stripslashes($_POST['email']) ?? ''), ENT_QUOTES, 'UTF-8');
$password = $_POST['password'] ?? '';
$confirmPassword = $_POST['confirm_password'] ?? '';

// Re-validate server-side
if (!isValidUsername($username)) {
    $errors[] = 'Username must be 3-50 characters: letters, numbers, underscore only.';
}
if ($name === '' || strlen($name) > 100) {
    $errors[] = 'Name is required and must be 100 characters or fewer.';
}
if (!isValidEmail($email)) {
    $errors[] = 'A valid email address is required.';
}
if (!isValidPassword($password)) {
    $errors[] = 'Password must be 8-50 characters, with at least 1 uppercase letter, 1 lowercase letter, 1 number, and 1 special character.';
}
if ($password !== $confirmPassword) {
    $errors[] = 'Passwords do not match.';
}

// check if someone already took this username before we insert
if (empty($errors)) {
    $stmt = $mysqli->prepare("SELECT username FROM users WHERE username = ?");
    $stmt->bind_param("s", $username);
    $stmt->execute();
    $stmt->store_result();
    if ($stmt->num_rows > 0) {
        $errors[] = 'That username is already taken. Please choose another.';
    }
    $stmt->close();
}

```



```

if (empty($errors)) {
    $hashedPassword = password_hash($password, PASSWORD_DEFAULT);

    $stmt = $mysqli->prepare("INSERT INTO users (username, password, name, email) VALUES (?, ?, ?, ?)");
    $stmt->bind_param("ssss", $username, $hashedPassword, $name, $email);

    if ($stmt->execute()) {
        $stmt->close();
        $mysqli->close();
        header("Location: index.php?registered=1");
        exit();
    } else {
        $errors[] = 'Registration failed. Please try again.';
    }
    $stmt->close();
}

$mysqli->close();
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WAPH - Registration Error</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Registration Failed</h1>
        <ul class="error">
            <?php foreach ($errors as $err): ?>
                <li><?php echo htmlspecialchars($err, ENT_QUOTES, 'UTF-8'); ?></li>
            <?php endforeach; ?>
        </ul>
        <p><a href="registrationform.php">Back to registration form</a>.</p>
    </div>
</body>
</html>

```

profile.php

```

<?php
require 'session_auth.php';
require 'db_connect.php';

```

```

$username = $_SESSION['username'];
$errors = [];
$success = isset($_GET['password_changed']) ? 'Password updated successfully.' : "";

// make a csrf token if we dont got one yet
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(openssl_random_pseudo_bytes(16));
}

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $csrfToken = $_POST['csrf_token'] ?? '';

    if (!hash_equals($_SESSION['csrf_token'], $csrfToken)) {
        $errors[] = 'Invalid or expired form submission. Please try again.';
    } else {
        $name = htmlspecialchars(trim(stripslashes($_POST['name']) ?? ''), ENT_QUOTES, 'UTF-8');
        $email = htmlspecialchars(trim(stripslashes($_POST['email']) ?? ''), ENT_QUOTES, 'UTF-8');

        if ($name === '' || strlen($name) > 100) {
            $errors[] = 'Name is required and must be 100 characters or fewer.';
        }
        if (filter_var($email, FILTER_VALIDATE_EMAIL) === false || strlen($email) > 100) {
            $errors[] = 'A valid email address is required.';
        }

        // only touching name and email here, nothing else changes
        if (empty($errors)) {
            $stmt = $mysqli->prepare("UPDATE users SET name = ?, email = ? WHERE username = ?");
            $stmt->bind_param("sss", $name, $email, $username);
            $stmt->execute();
            $stmt->close();
            $success = 'Profile updated successfully.';
        }
    }
}

$stmt = $mysqli->prepare("SELECT name, email FROM users WHERE username = ?");
$stmt->bind_param("s", $username);
$stmt->execute();
$stmt->bind_result($name, $email);
$stmt->fetch();
$stmt->close();
$mysqli->close();
?>
<!DOCTYPE html>

```

```

<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WAPH - Profile</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        
        <h1>Welcome, <?php echo htmlentities($username, ENT_QUOTES, 'UTF-8'); ?></h1>

        <?php if (!empty($errors)): ?>
            <ul class="error">
                <?php foreach ($errors as $err): ?>
                    <li><?php echo htmlspecialchars($err, ENT_QUOTES, 'UTF-8'); ?></li>
                <?php endforeach; ?>
            </ul>
        <?php endif; ?>
        <?php if ($success !== ''): ?>
            <p class="success"><?php echo htmlspecialchars($success, ENT_QUOTES, 'UTF-8'); ?>
        <?php endif; ?>

        <form method="POST" action="profile.php" class="form profile">
            <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8'); ?>" />
            <label>Name:
                <input type="text" class="text_field" name="name" required maxlength="100"
                    value="<?php echo htmlentities($name, ENT_QUOTES, 'UTF-8'); ?>" />
            </label>
            <label>Email:
                <input type="email" class="text_field" name="email" required maxlength="100"
                    value="<?php echo htmlentities($email, ENT_QUOTES, 'UTF-8'); ?>" />
            </label>
            <button class="button" type="submit">Save Changes</button>
        </form>

        <p><a href="changepasswordform.php">Change Password</a> | <a href="logout.php">Logout</a>
    </div>
</body>
</html>

```

changepasswordform.php

```

<?php
require 'session_auth.php';

// need a token here too to prevent people from changing passwords without knowing

```

```

if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(openssl_random_pseudo_bytes(16));
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>WAPH - Change Password</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Change Password</h1>

        <?php if (isset($_GET['error'])): ?>
            <p class="error"><?php echo htmlspecialchars($_GET['error'], ENT_QUOTES, 'UTF-8')>
        <?php endif; ?>

        <form method="POST" action="changepassword.php" class="form change-password" onsubmit="return checkPasswordsMatch();">
            <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8')>" />
            <!-- same password rules as the register page, gotta stay consistent -->
            <label>New Password:
                <input type="password" class="text_field" name="new_password" id="new_password"
                    required maxlength="50"
                    pattern="(?!.*[a-z])(?!.*[A-Z])(?!.*\d)(?!.*[^a-zA-Z0-9]).{8,50}"
                    title="At least 8 characters, with 1 uppercase, 1 lowercase, 1 number" />
            </label>
            <label>Confirm New Password:
                <input type="password" class="text_field" name="confirm_password" id="confirm_password" />
            </label>
            <p class="error" id="match-error" style="display:none;">Passwords do not match.
            <button class="button" type="submit">Update Password</button>
        </form>

        <p><a href="profile.php">Back to profile</a></p>
    </div>

    <script>
        // same match check as registration
        function checkPasswordsMatch()
        {
            const password = document.getElementById('new_password').value;
            const confirm = document.getElementById('confirm_password').value;
            const errorEl = document.getElementById('match-error');
            if (password !== confirm) {

```

```

        errorEl.style.display = 'block';
        return false;
    }
    errorEl.style.display = 'none';
    return true;
}
</script>
</body>
</html>

```

changepassword.php

```

<?php
require 'session_auth.php';

$username = $_SESSION['username'];
$csrfToken = $_POST['csrf_token'] ?? '';

// checking the token before touching the password at all
if (!hash_equals($_SESSION['csrf_token'] ?? '', $csrfToken)) {
    header("Location: changepasswordform.php?error=" . urlencode('Invalid or expired form session token'));
    exit();
}

$newPassword = $_POST['new_password'] ?? '';
$confirm_password = $_POST['confirm_password'] ?? '';

// same rules as registration so its consistent everywhere
if (!preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[~a-zA-Z0-9]).{8,50}$/', $newPassword)) {
    header("Location: changepasswordform.php?error=" . urlencode('Password must be 8-50 characters long and contain at least one lowercase letter, one uppercase letter, one digit, and one special character'));
    exit();
}

if ($newPassword !== $confirm_password) {
    header("Location: changepasswordform.php?error=" . urlencode('Passwords do not match.'));
    exit();
}

require 'db_connect.php';

// bcrypt hash, never ever store the plain password
$hashedPassword = password_hash($newPassword, PASSWORD_DEFAULT);

$stmt = $mysqli->prepare("UPDATE users SET password = ? WHERE username = ?");
$stmt->bind_param("ss", $hashedPassword, $username);
$stmt->execute();

```

```

$stmt->close();
$mysqli->close();

header("Location: profile.php?password_changed=1");
exit();

```

session_auth.php

```

<?php
// put at top of any page that needs a login to view

session_set_cookie_params([
    'lifetime' => 1800,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

session_start();

if (isset($_SESSION['authenticated']) && $_SESSION['authenticated'] === true) {
    if (!isset($_SESSION['user_agent']) || $_SESSION['user_agent'] !== $_SERVER['HTTP_USER_AGENT']) {
        $_SESSION = array();
        session_destroy();
        header("Location: index.php");
        exit();
    }
}
else {
    // if no session then they dont get to see the page
    header("Location: index.php");
    exit();
}

```

logout.php

```

<?php
session_set_cookie_params([
    'lifetime' => 1800,
    'path' => '/',
    'domain' => '',
    'secure' => true,
    'httponly' => true,
    'samesite' => 'Strict'
]);

```

```

]);

session_start();

// wiping session data out first
$_SESSION = array();

if (ini_get("session.use_cookies")) {
    // killing the cookie in the browser too so nothing sticks around
    $params = session_get_cookie_params();
    setcookie(session_name(), "", time() - 42000,
        $params["path"], $params["domain"],
        $params["secure"], $params["httponly"]
    );
}

session_destroy();

header("Location: index.php");
exit();

```

db__connect.php

```

<?php
// using the non root user from lab 3, dont wanna give the app full db access
$mysqli = new mysqli('localhost', 'ward3aj', 'Password123', 'waph');

// if this breaks its probably the password or db name being wrong
if ($mysqli->connect_errno) {
    die("Database connection failed: " . $mysqli->connect_error);
}

```

database-data.sql

```

-- resetting users table
USE waph;

drop table if exists users;

CREATE TABLE users (
    username VARCHAR(50) PRIMARY KEY,
    password VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);

```

```

-- test logins, plaintext passwords are down here if u forget them
-- admin -> Password123!  aiden -> Aiden123!
INSERT INTO users (username, password, name, email) VALUES
('admin', '$2y$12$gwWSJkDiupFq8wl6cPHS8uJeJUKv7D2iCC0drfchr6TTt1hTXjlQ.', 'Admin User', 'adm
('aiden', '$2y$12$HiDK3kYbhWwlsu0FGJ7nm.92m9QxCGHMgIofw8BKD2WTDH59t9WUG', 'Aiden Ward', 'war

```

style.css

```

* {
    font-family: 'Inter', sans-serif !important;
}

body {
    padding-top: 20px;
    padding-bottom: 50px;
    transition: background-color 0.3s, color 0.3s;
}

.company-logo {
    width: 40px !important;
    height: 40px !important;
    object-fit: contain !important;
    margin-right: 15px;
    background-color: white;
    border-radius: 4px;
}

/* dark mode toggle stuff */
.dark-mode {
    background-color: #1a1a1a !important;
    color: #f1f1f1 !important;
}

.dark-mode .bg-light {
    background-color: #333 !important;
    color: #f1f1f1 !important;
    border-color: #444 !important;
}

.dark-mode .text-muted {
    color: #aaa !important;
}

/* everything below here is the project 2 form styling */

.container {

```



```

    max-width: 480px;
    margin: 0 auto;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 8px;
}

.form {
    display: flex;
    flex-direction: column;
}

.form label,
.form .text_field {
    margin-bottom: 12px;
}

.text_field {
    padding: 8px;
    border: 1px solid #ccc;
    border-radius: 4px;
    width: 100%;
    box-sizing: border-box;
}

.button {
    padding: 10px;
    background-color: #333;
    color: #fff;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    width: 100%;
}

.button:hover { background-color: #555; }

.error {
    color: #b00020;
    font-weight: bold;
}

.success {
    color: #1a7a1a;
    font-weight: bold;
}

```

```
.headshot-small {  
  width: 60px;  
  height: 60px;  
  border-radius: 50%;  
  object-fit: cover;  
}
```